

Classificazione Funzionale di Strutture Proteiche tramite Reti Neurali Graph Convolutional: un'Analisi Comparativa ed Empirica

Edoardo Galantini¹

¹e.galantini1@campus.uniurb.it

Riassunto

La classificazione funzionale delle proteine rappresenta un problema di particolare interesse nell'ambito della bioinformatica e del Deep Learning applicato ai dati strutturali. In questo lavoro viene presentata un'analisi empirica e comparativa sull'efficacia delle Graph Neural Networks (GNN) nel compito di classificazione binaria di strutture proteiche (enzimi versus non-enzimi) utilizzando il dataset perturbato G3, derivato da PROTEINS_full [1]. Lo studio è stato organizzato in due fasi principali. Nella prima è stata effettuata una Model Selection tramite Grid Search per confrontare le architetture Graph Convolutional Network (GCN) e Graph Attention Network (GAT) all'interno di uno schema di Stratified 3-Fold Cross-Validation. Nella seconda fase sono state valutate diverse strategie di preprocessing e ottimizzazione del modello, tra cui la trasformazione logaritmica delle feature con distribuzione asimmetrica, la rimozione delle feature One-Hot per analizzarne il contributo informativo e il tuning del numero di hidden channels. I risultati mostrano che la configurazione migliore è stata ottenuta con l'architettura ProteinGCN, utilizzando 64 hidden channels, learning rate pari a 10^{-3} e dropout di 0.3. Il modello finale, addestrato sull'intero training set con l'aggiunta di Noise Injection sui nodi per incrementare la robustezza generale, ha raggiunto sul test set indipendente un Macro F_1 -score di 0.7603 e una ROC-AUC di 0.8351, confermando l'efficacia delle GNN nell'apprendimento delle informazioni strutturali delle proteine per questo compito di classificazione.

1 Introduzione

La classificazione automatica delle proteine a partire dalla loro struttura tridimensionale rappresenta un problema di interesse nell'ambito della bioinformatica, poiché consente di supportare lo studio della funzione delle proteine attraverso strumenti computazionali. Negli ultimi anni, i metodi di Deep Learning hanno mostrato risultati promettenti in questo contesto, grazie alla loro capacità di apprendere automaticamente rappresentazioni significative dei dati senza richiedere una fase estesa di feature engineering.

Le proteine possono essere rappresentate come grafi, in cui i nodi rappresentano i residui amminocidici e gli archi descrivono le relazioni tra di essi. Questa rappresentazione rende le Graph Neural Networks (GNN) particolarmente adatte a questo tipo di problema.

In questo lavoro viene affrontato un problema di classificazione binaria di grafi utilizzando il dataset G3,

una versione perturbata del benchmark PROTEINS_full. L'obiettivo è determinare se una proteina appartenga alla classe degli enzimi ($y = 1$) oppure a quella delle proteine non enzimatiche ($y = 0$). Ogni nodo del grafo è descritto da un vettore di 32 feature, costituito da 29 attributi continui relativi alle proprietà dell'amminoacido e da 3 feature binarie ottenute tramite codifica One-Hot della struttura secondaria (α -elica, foglietto β e turn/coil).

Per garantire una valutazione corretta delle prestazioni del modello, il lavoro è stato organizzato nelle seguenti fasi:

1. **Suddivisione del Dataset:** divisione iniziale del dataset in modo stratificato, riservando il 15% dei dati come Test Set finale, utilizzato esclusivamente per la valutazione conclusiva del modello.
2. **Model Selection:** confronto tra le architetture Graph Convolutional Network (GCN) e Graph Attention Network (GAT) mediante una

Grid Search sui principali iperparametri (learning rate e dropout), utilizzando uno schema di Stratified 3-Fold Cross-Validation.

3. **Analisi delle Configurazioni:** analisi dell'effetto di alcune scelte di preprocessing e configurazione del modello, valutando l'impatto della trasformazione logaritmica delle feature continue, della rimozione delle feature One-Hot della struttura secondaria e della variazione del numero di hidden channels.

La relazione è organizzata come segue. La Sezione 2 introduce i principali concetti teorici relativi alle Graph Neural Networks e alle funzioni di perdita utilizzate. La Sezione 3 descrive il dataset, la pipeline sperimentale e le metriche di valutazione adottate. La Sezione 4 presenta e discute i risultati ottenuti nei diversi esperimenti, mentre la Sezione 5 riassume le conclusioni del lavoro e propone alcuni possibili sviluppi futuri.

2 Metodi

In questa sezione vengono descritti i modelli e gli algoritmi di apprendimento utilizzati per la classificazione dei grafi proteici. Vengono formalizzati i meccanismi di propagazione dei messaggi delle due architetture a confronto (GCN e GAT), la strategia di aggregazione spaziale (Readout) e la funzione di perdita ottimizzata durante il training.

2.1 Graph Neural Networks e Message Passing

Il principio fondamentale delle Graph Neural Networks (GNN) risiede nel paradigma del *Message Passing*. In ogni strato della rete, la rappresentazione (o embedding) di ciascun nodo viene aggiornata combinando le informazioni provenienti dal nodo stesso con quelle aggregate dal suo vicinato locale. Per un generico strato k , l'aggiornamento dello stato del nodo u può essere espresso formalmente come:

$$h_u^{(k+1)} = \text{UPDATE}^{(k)}\left(h_u^{(k)}, \text{AGGREGATE}^{(k)}\left(\{h_v^{(k)} : v \in \mathcal{N}(u)\}\right)\right) \quad (1)$$

dove $\mathcal{N}(u)$ rappresenta l'insieme dei nodi adiacenti (vicini) al nodo u , mentre AGGREGATE e UPDATE sono funzioni differenziabili implementate dai diversi strati convolutivi.

2.2 Graph Convolutional Network (GCN)

L'architettura **ProteinGCN** utilizza strati di tipo **GCNConv**. In questo modello, l'operazione di convoluzione su grafo applica una media pesata delle feature dei vicini, introducendo una normalizzazione simmetrica basata sul grado dei nodi per evitare che nodi con un'elevata connettività destabilizzino la scala delle attivazioni. La regola di aggiornamento del nodo u allo strato k è definita considerando l'aggiunta di un auto-anello (*self-loop*) per includere le feature del nodo stesso nel calcolo, dove i pesi condivisi sono rappresentati dalla matrice apprendibile $W^{(k)}$ e σ rappresenta la funzione di attivazione non lineare ReLU. In formula:

$$h_u^{(k)} = \sigma \left(W^{(k)} \sum_{v \in \mathcal{N}(u) \cup \{u\}} \frac{h_v^{(k-1)}}{\sqrt{\tilde{d}_u \tilde{d}_v}} \right) \quad (2)$$

dove $\tilde{d}_u = |\mathcal{N}(u)| + 1$ rappresenta il grado del nodo u aumentato per includere il *self-loop*.

Come implementato nel modulo `models.py`, l'architettura prevede due strati consecutivi di **GCNConv** per estendere il campo ricettivo della rete fino ai vicini di secondo ordine, mantenendo costante la dimensione dello spazio latente a un numero fisso di canali nascosti (*hidden channels*).

2.3 Graph Attention Network (GAT)

L'architettura **ProteinGAT** sostituisce la normalizzazione fissa basata sui gradi con un meccanismo di attenzione tramite strati **GATConv**. Questo approccio permette alla rete di computare dinamicamente un coefficiente di importanza per ogni arco, stabilendo quanto l'informazione di un nodo vicino sia rilevante per il nodo destinatario. La formulazione del singolo strato di attenzione è la seguente:

$$h_u^{(k)} = \sigma \left(\sum_{v \in \mathcal{N}(u) \cup \{u\}} \alpha_{u,v} \mathbf{W}^{(k)} \mathbf{h}_v^{(k-1)} \right) \quad (3)$$

I coefficienti di attenzione $\alpha_{u,v}$ vengono ottenuti normalizzando gli score di attenzione non normalizzati $e_{u,v}$ su tutto il vicinato mediante la funzione Softmax:

$$\alpha_{u,v} = \frac{\exp(e_{u,v})}{\sum_{v' \in \mathcal{N}(u)} \exp(e_{u,v'})} \quad (4)$$

dove il coefficiente $e_{u,v}$ rappresenta lo score di attenzione grezzo (*raw attention score*) che esprime l'importanza del nodo v per l'aggiornamento del nodo u , ed è calcolato come:

$$e_{u,v} = \text{LeakyReLU} \left(\mathbf{a}_k^T (\mathbf{W}_k \mathbf{h}_u^{(k-1)} \parallel \mathbf{W}_k \mathbf{h}_v^{(k-1)}) \right) \quad (5)$$

In quest'ultima espressione, il simbolo $\parallel$ denota l'operazione di concatenazione dei vettori delle feature.

2.4 Fase di Readout e Pooling Globale

Poiché il task richiede la classificazione dell'intero grafo proteico e non dei singoli nodi, è necessario convertire la matrice delle feature dei nodi, le cui dimensioni dipendono dal numero variabile di amminoacidi della specifica proteina, in un vettore a dimensione fissa. Questo processo prende il nome di *READOUT*.

Per catturare sia le proprietà medie della macromolecola sia le caratteristiche locali più salienti espresse dai picchi di attivazione, si è scelto di combinare due operatori di pooling globali deterministici attraverso la concatenazione:

$$z_G = \left[\frac{1}{|V|} \sum_{u \in V} h_u^{(K)} \parallel \max_{u \in V} h_u^{(K)} \right] \quad (6)$$

Il vettore risultante z_G ha una dimensionalità doppia rispetto all'incorporamento dei singoli nodi ed è indipendente dall'ordine di questi ultimi. L'operazione di aggregazione su un insieme non ordinato garantisce infatti la proprietà di invarianza alle permutazioni, fondamentale per l'elaborazione dei grafi a livello globale.

2.5 Classificatore e Funzione di Loss

Il vettore del grafo z_G viene infine passato a un classificatore composto da un Percettrone Multistrato (MLP) a due livelli lineari intervallati da un'attivazione ReLU e da uno strato di Dropout. Quest'ultimo azzerava casualmente una frazione delle connessioni durante il training, così che una sottorete differente sia attiva ad ogni forward pass, incrementando la stabilità, la ridondanza e prevenendo l'overfitting.

L'ultimo livello lineare proietta lo spazio latente in un singolo valore numerico continuo, il *logit* (punteggio grezzo). Il problema di classificazione binaria viene ottimizzato mediante la funzione di perdita **BCEWithLogitsLoss**. Come raccomandato dalle best practice, questo modulo viene preferito all'applicazione manuale della Sigmoidale seguita da una loss binaria standard, in quanto integra internamente l'applicazione della funzione di attivazione Sigmoidale direttamente ai logit previsti dal modello, confrontandoli poi con le etichette reali.

3 Configurazione sperimentale

In questa sezione vengono descritte le principali caratteristiche del dataset utilizzato, le modalità di suddivisione dei dati, la pipeline di preprocessing, gli iperparametri considerati durante la fase di Model Selection e le scelte adottate per garantire la riproducibilità degli esperimenti.

3.1 Analisi e Caratteristiche del Dataset

Gli esperimenti sono stati condotti sul dataset G3, una versione perturbata di **PROTEINS_full**. Ogni campione del dataset rappresenta una proteina modellata come un grafo, mentre il compito consiste nel classificare ciascun grafo come enzima (classe 1) oppure non enzima (classe 0).

Il dataset contiene complessivamente 1001 grafi. Un'analisi preliminare della distribuzione delle classi, effettuata tramite la funzione **check_class_imbalance**, ha evidenziato la seguente ripartizione:

- **Classe 0 (Non-Enzimi):** 589 grafi (58.84%);
- **Classe 1 (Enzimi):** 412 grafi (41.16%).

Lo sbilanciamento tra le due classi risulta moderato e non richiede particolari interventi di preprocessing.

Ogni nodo del grafo è descritto da un vettore di 32 feature. In particolare, sono presenti 29 feature continue associate alle proprietà dell'amminoacido e 3 feature binarie generate automaticamente dalla libreria **PyTorch Geometric** tramite codifica One-Hot della struttura secondaria (α -elica, foglietto β e turn/coil).

3.2 Partizionamento dei Dati e Validazione

Per evitare fenomeni di *data leakage*, il dataset è stato suddiviso fin dall'inizio in modo stratificato mediante la funzione **stratified_holdout_split**, utilizzando il parametro **stratify=labels**. Il 15% dei dati (151 grafi) è stato riservato come **Test Set finale**, mantenendo la distribuzione originaria delle classi. Questo insieme di dati non è mai stato utilizzato durante l'addestramento o la selezione degli iperparametri ed è stato impiegato esclusivamente per la valutazione finale del modello.

I restanti 850 grafi (85%) costituiscono il set di training e validazione. Su questo insieme è stata applicata una Stratified 3-Fold Cross-Validation (**Stratified KFold**), scelta per ottenere una stima affidabile delle prestazioni mantenendo tempi di addestramento contenuti.

3.3 Pipeline di Preprocessing e Configurazione della Griglia

Per ogni fold della Cross-Validation, tutte le operazioni di preprocessing sono state eseguite utilizzando esclusivamente i dati del training fold. Le statistiche calcolate sono state successivamente applicate anche al corrispondente validation fold, evitando qualsiasi contaminazione tra i dati.

La pipeline di preprocessing è composta dalle seguenti fasi:

1. **Normalizzazione Z-Score:** la media e la deviazione standard sono state calcolate sulle sole 29 feature continue del training fold. La normalizzazione è stata applicata esclusivamente a queste feature, lasciando inalterate le 3 feature One-Hot della struttura secondaria e la feature discreta all'indice 4.
2. **Noise Injection:** durante l'addestramento è stato aggiunto rumore gaussiano di intensità pari a 0.01 alle feature continue dei nodi del training fold. Questa tecnica è stata utilizzata per migliorare la capacità di generalizzazione del modello e ridurre il rischio di overfitting.

Durante la fase di Grid Search sono stati esplorati i seguenti valori degli iperparametri per confrontare le architetture ProteinGCN e ProteinGAT:

- **Learning Rate:** $[10^{-3}, 10^{-4}]$;
- **Dropout:** $[0.1, 0.3]$.

Per tutti gli esperimenti sono stati utilizzati l'ottimizzatore Adam, un valore di `weight_decay` pari a 10^{-4} , una `batch_size` di 32, 50 epoche di addestramento e un numero iniziale di `hidden_channels` pari a 64.

3.4 Riproducibilità degli Esperimenti

Per garantire la riproducibilità dei risultati, il progetto è stato organizzato in moduli eseguibili direttamente tramite il comando `python main.py`. Inoltre, nel file `utils.py` sono state adottate le seguenti impostazioni:

- utilizzo di un seme casuale fisso (`FIXED_SEED = 42`) per Python, NumPy e PyTorch;
- attivazione degli algoritmi deterministici di PyTorch tramite:
`torch.use_deterministic_algorithms(True)`;
- limitazione del numero di thread di esecuzione a uno (`torch.set_num_threads(1)`);
- utilizzo esclusivo della CPU (`device = 'cpu'`).

4 Risultati sperimentali

In questa sezione vengono presentati i risultati ottenuti nelle diverse fasi del progetto. In particolare, vengono discussi i risultati della Grid Search, gli esperimenti svolti durante la Cross-Validation e le prestazioni finali del modello sul Test Set.

4.1 Model Selection (Grid Search)

La prima fase della sperimentazione ha riguardato il confronto tra le due architetture considerate, ProteinGCN e ProteinGAT, variando il learning rate (lr) e il dropout. I modelli sono stati valutati mediante Stratified 3-Fold Cross-Validation utilizzando come metrica principale il Macro F_1 -score e, come metrica aggiuntiva, la ROC-AUC.

I risultati della Grid Search sono riportati nella Tabella 1.

Dall'analisi dei risultati emerge che l'architettura ProteinGCN ottiene le prestazioni migliori nella configurazione `GCN_lr_0.001_drop_0.3`, raggiungendo un Macro F_1 -score medio pari a 0.7393. Questa configurazione presenta inoltre la deviazione standard più bassa (± 0.0021), indicando una buona stabilità delle prestazioni nei diversi fold della Cross-Validation. Le configurazioni basate su ProteinGAT mostrano invece una variabilità maggiore tra i fold. Per questo motivo, la configurazione migliore della GCN viene scelta come modello di riferimento per gli esperimenti successivi.

4.2 Esperimenti Incrementali

Dopo aver individuato la configurazione migliore, sono stati eseguiti tre esperimenti per valutare l'effetto di alcune scelte di preprocessing e configurazione del modello.

4.2.1 Test 1: Trasformazione Logaritmica

L'analisi preliminare delle feature continue (grafico 1) ha evidenziato che alcune variabili presentano una distribuzione fortemente asimmetrica. Per questo motivo è stata valutata l'applicazione di una trasformazione logaritmica calcolata utilizzando esclusivamente il training fold.

Il confronto dei risultati, riportato nel grafico `test_log_transform.png`, mostra che:

- **Con Log-Transform:** Macro $F_1 = 0.7325$ | ROC-AUC = 0.8168
- **Senza Log-Transform:** Macro $F_1 = 0.7393$ | ROC-AUC = 0.7922

Tabella 1: Graduatoria delle configurazioni testate in Cross-Validation.

Configurazione	Macro F_1	Dev. Std	ROC-AUC	Dev. Std
GCN_lr_0.001_drop_0.3	0.7393	0.0021	0.7922	0.0091
GAT_lr_0.001_drop_0.3	0.7309	0.0192	0.7963	0.0308
GCN_lr_0.0001_drop_0.3	0.7296	0.0253	0.7973	0.0162
GAT_lr_0.0001_drop_0.1	0.7272	0.0223	0.8002	0.0325
GCN_lr_0.001_drop_0.1	0.7223	0.0166	0.7854	0.0089
GCN_lr_0.0001_drop_0.1	0.7185	0.0282	0.7888	0.0155
GAT_lr_0.001_drop_0.1	0.7090	0.0127	0.7864	0.0282
GAT_lr_0.0001_drop_0.3	0.7074	0.0274	0.8019	0.0264

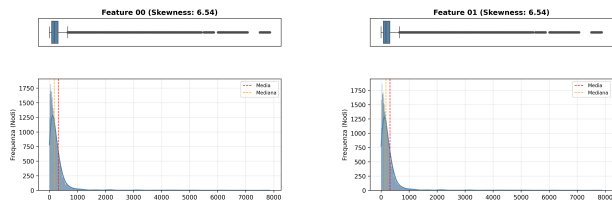


Figura 1: Grafico delle prime due feature che presentano forte asimmetria

La trasformazione logaritmica migliora la ROC-AUC, ma comporta una leggera diminuzione del Macro F_1 -score, metrica scelta come riferimento principale. Per questo motivo la trasformazione non viene utilizzata nella configurazione finale del modello.

4.2.2 Test 2: Esclusione delle Feature One-Hot

Il secondo esperimento valuta l'effetto della rimozione delle tre feature One-Hot relative alla struttura secondaria. Come mostra la figura 2, i risultati ottenuti sono i seguenti:

- **Modello Base (32 feature):** Macro $F_1 = 0.7393$ | ROC-AUC = 0.7922
- **Modello senza One-Hot (29 feature):** Macro $F_1 = 0.7314$ | ROC-AUC = 0.8004

La rimozione delle feature One-Hot determina una riduzione del Macro F_1 -score. Questo risultato suggerisce che tali feature forniscono informazioni utili al modello e contribuiscono a migliorare la classificazione. Di conseguenza, vengono mantenute nella configurazione finale.

4.2.3 Test 3: Numero di Hidden Channels

L'ultimo esperimento confronta due configurazioni del modello con un diverso numero di hidden channels: 64 e 32.

- **64 hidden channels:** Macro $F_1 = 0.7393$ | ROC-AUC = 0.7922

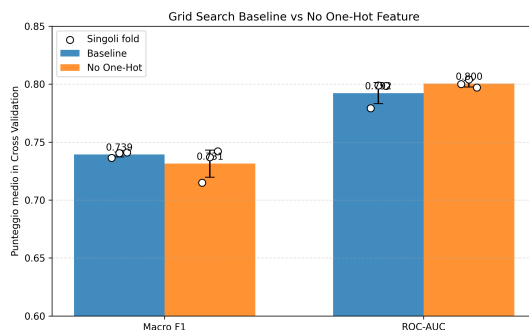


Figura 2: Confronto delle performance tra la Baseline e la rimozione delle feature One-Hot.

- **32 hidden channels:** Macro $F_1 = 0.7271$ | ROC-AUC = 0.8078

La configurazione con 32 hidden channels ottiene un Macro F_1 -score inferiore rispetto a quella con 64. Questo risultato mostra che la configurazione con 64 hidden channels offre prestazioni migliori rispetto a quella con 32. Per questo motivo, viene mantenuta nel modello finale.

4.3 Valutazione sul Test Set Finale

Sulla base dei risultati ottenuti durante la Cross-Validation, il modello finale è stato configurato utilizzando l'architettura **ProteinGCN**, learning rate pari a 10^{-3} , dropout pari a 0.3 e 64 hidden channels. Successivamente è stato addestrato sull'intero Training Set (850 grafi) e valutato sul Test Set indipendente composto da 151 grafi.

Le metriche ottenute sono le seguenti:

- **Accuracy:** 0.7616
- **Macro F_1 -score:** 0.7603
- **Precision:** 0.6667
- **Recall:** 0.8387
- **ROC-AUC:** 0.8351

L'andamento della Training Loss, riportato nella Figura 3, mostra una diminuzione regolare durante l'ad-

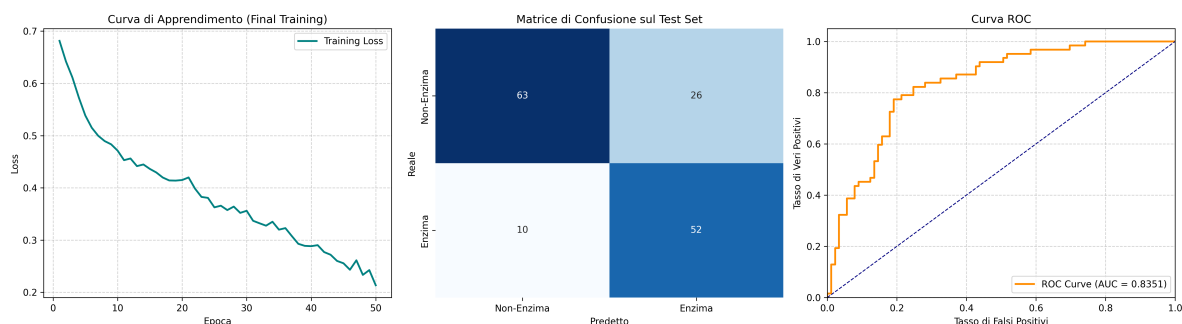


Figura 3: Grafico di valutazione finale: Curva di Apprendimento (sinistra), Matrice di Confusione sul Test Set (centro) e Curva ROC (destra).

destramento, senza oscillazioni significative. Inoltre, il Macro F_1 -score ottenuto sul Test Set risulta leggermente superiore a quello osservato durante la Cross-Validation (0.7603 contro 0.7393), suggerendo che le tecniche di regolarizzazione adottate, come il Dropout, il weight decay e la Noise Injection, abbiano favorito una buona capacità di generalizzazione del modello.

Infine, la matrice di confusione e la curva ROC confermano le buone prestazioni del modello. In particolare, il Recall pari a 0.8387 indica che il modello riesce a identificare correttamente la maggior parte degli esempi appartenenti alla classe degli enzimi.

5 Conclusioni

In questo lavoro è stata presentata un'analisi empirica e comparativa sull'utilizzo delle Graph Neural Networks per un problema di classificazione binaria di proteine, utilizzando il dataset perturbato G3, derivato da PROTEINS_full [1]. Il progetto ha permesso di sviluppare e valutare un'intera pipeline di Deep Learning, comprendendo le fasi di preprocessing, selezione del modello, validazione tramite Cross-Validation e valutazione finale su un Test Set indipendente.

I risultati ottenuti mostrano che l'architettura ProteinGCN, configurata con learning rate pari a 10^{-3} , dropout di 0.3 e 64 hidden channels, rappresenta la soluzione più efficace tra quelle considerate. Rispetto alla ProteinGAT, la GCN ha ottenuto il miglior Macro F_1 -score medio (0.7393) e ha mostrato una maggiore stabilità tra i diversi fold della Cross-Validation, come evidenziato dalla ridotta deviazione standard (± 0.0021).

Gli esperimenti aggiuntivi hanno permesso di valutare l'impatto di alcune scelte progettuali. La trasformazione logaritmica delle feature continue ha migliorato la ROC-AUC, ma ha comportato una leggera

riduzione del Macro F_1 -score, motivo per cui non è stata adottata nella configurazione finale. La rimozione delle feature One-Hot ha determinato un peggioramento delle prestazioni, suggerendo che tali informazioni contribuiscano al processo di classificazione. Infine, il confronto tra 32 e 64 hidden channels ha evidenziato come quest'ultima configurazione permetta di ottenere risultati migliori.

Il modello finale, addestrato sull'intero Training Set e valutato sul Test Set indipendente, ha raggiunto un Macro F_1 -score pari a 0.7603 e una ROC-AUC pari a 0.8351. Nel complesso, i risultati ottenuti suggeriscono che una Graph Convolutional Network relativamente semplice, opportunamente configurata e accompagnata da tecniche di regolarizzazione come Dropout, weight decay e Noise Injection, sia in grado di affrontare efficacemente questo problema di classificazione.

In un'ottica di sviluppo futuro, la pipeline potrebbe essere estesa sfruttando feature associate agli archi del grafo (matrice `edge_attr`), qualora fossero disponibili, per verificare se possano migliorare le prestazioni del modello. Inoltre, si potrebbe valutare l'applicazione del Transfer Learning, pre-addestrando l'architettura su un database più grande (dominio sorgente) per poi effettuare il fine-tuning specifico sul Dataset G3. Infine, anche l'impiego di tecniche di pre-addestramento auto-supervisionato potrebbe rappresentare un'opzione utile per migliorare l'inizializzazione delle rappresentazioni apprese dal modello.

Riferimenti bibliografici

- [1] Christopher Morris et al. "TUDataset: A collection of benchmark datasets for learning with graphs". In: *ICML 2020 Workshop on Graph Representation Learning and Beyond (GRL+ 2020)*. 2020. arXiv: 2007.08663. URL: www.graphlearning.io.